

Name: Muhammad Shayan Asad

Roll No: 23I-0518

Section: A

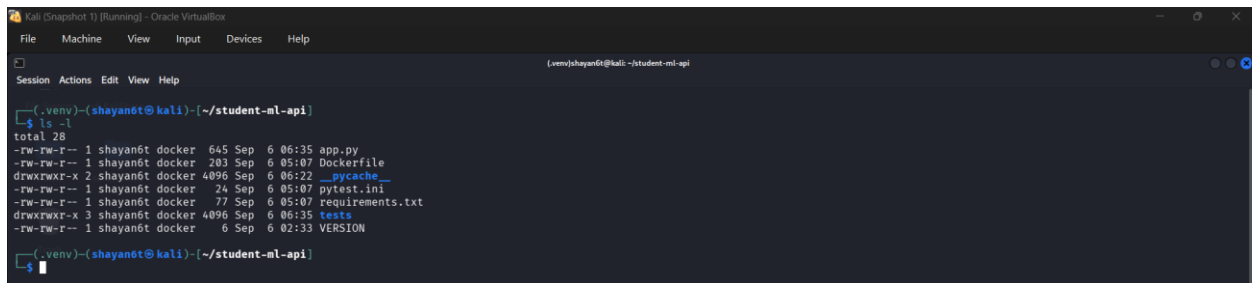
MLOps Assignment 01

GitHub Repository Link: <https://github.com/Shayan6t/student-ml-api>

Tasks Report:

Part 01 & 02: Setup and Testing

Figure 1: Initial project structure and essential repository files.

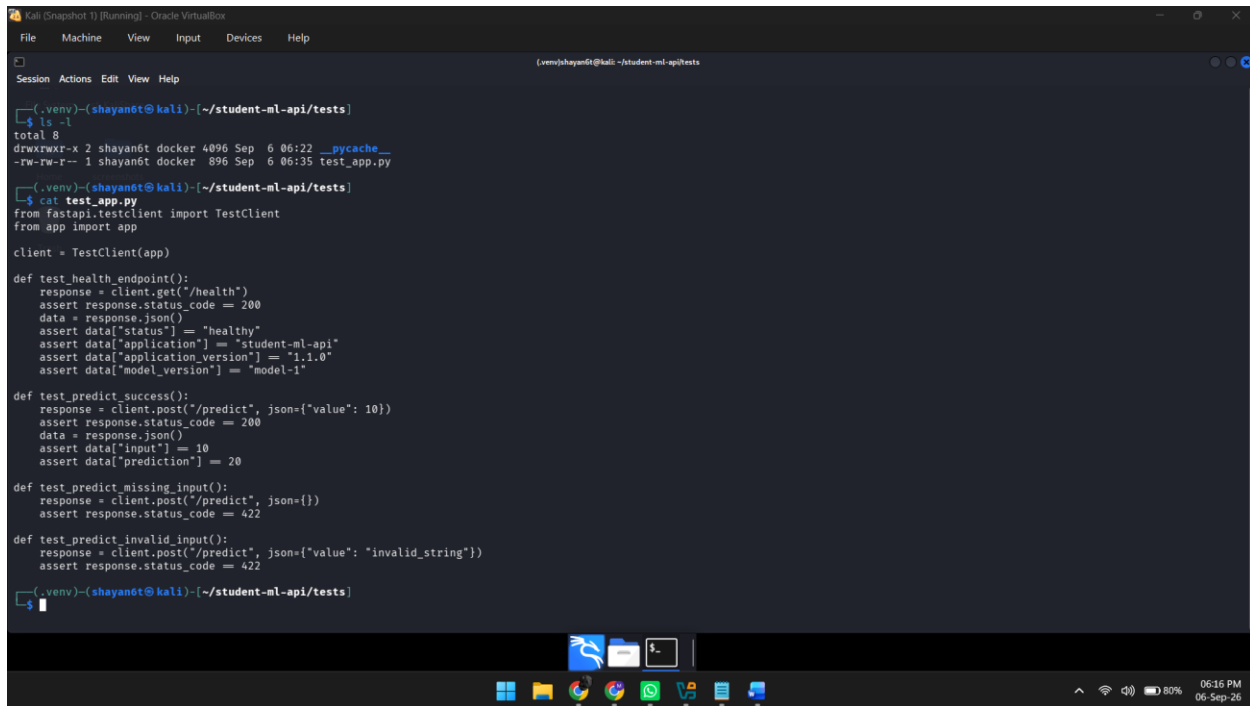


```
Kali (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

(.venv)-(shayan6t@kali)-[~/student-ml-api]
└─$ ls -l
total 28
-rw-rw-r-- 1 shayan6t docker 645 Sep  6 06:35 app.py
-rw-rw-r-- 1 shayan6t docker 203 Sep  6 05:07 Dockerfile
drwxrwxr-x 2 shayan6t docker 4096 Sep  6 06:22 __pycache__
-rw-rw-r-- 1 shayan6t docker 24 Sep  6 05:07 pytest.ini
-rw-rw-r-- 1 shayan6t docker 77 Sep  6 05:07 requirements.txt
drwxrwxr-x 3 shayan6t docker 4096 Sep  6 06:35 tests
-rw-rw-r-- 1 shayan6t docker  6 Sep  6 02:33 VERSION

(.venv)-(shayan6t@kali)-[~/student-ml-api]
└─$
```

Figure 2: Pytest suite defining API endpoint test cases.



```
Kali (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
(.venv)shayan6t@kali: ~/student-ml-api/tests
$ ls -l
total 8
drwxrwxr-x 2 shayan6t docker 4096 Sep  6 06:22 __pycache__
-rw-rw-r-- 1 shayan6t docker 896 Sep  6 06:35 test_app.py
(.venv)shayan6t@kali: ~/student-ml-api/tests
$ cat test_app.py
from fastapi.testclient import TestClient
from app import app

client = TestClient(app)

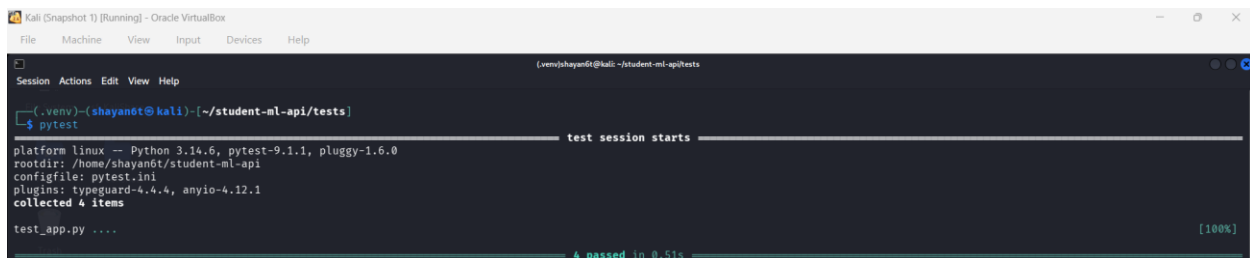
def test_health_endpoint():
    response = client.get("/health")
    assert response.status_code == 200
    data = response.json()
    assert data["status"] == "healthy"
    assert data["application"] == "student-ml-api"
    assert data["application_version"] == "1.1.0"
    assert data["model_version"] == "model-1"

def test_predict_success():
    response = client.post("/predict", json={"value": 10})
    assert response.status_code == 200
    data = response.json()
    assert data["input"] == 10
    assert data["prediction"] == 20

def test_predict_missing_input():
    response = client.post("/predict", json={})
    assert response.status_code == 422

def test_predict_invalid_input():
    response = client.post("/predict", json={"value": "invalid_string"})
    assert response.status_code == 422
(.venv)shayan6t@kali: ~/student-ml-api/tests
$
```

Figure 3: Local unit test execution verifying all endpoints pass.



```
Kali (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
(.venv)shayan6t@kali: ~/student-ml-api/tests
$ pytest
platform linux -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/shayan6t/student-ml-api
configfile: pytest.ini
plugins: typeguard-4.4.4, anyio-4.12.1
collected 4 items

test_app.py .... [100%]

4 passed in 0.51s
```

Parts 4–9: Branch Protection & CI Pipeline

Figure 4: Branch protection ruleset enforcing pull requests on main.

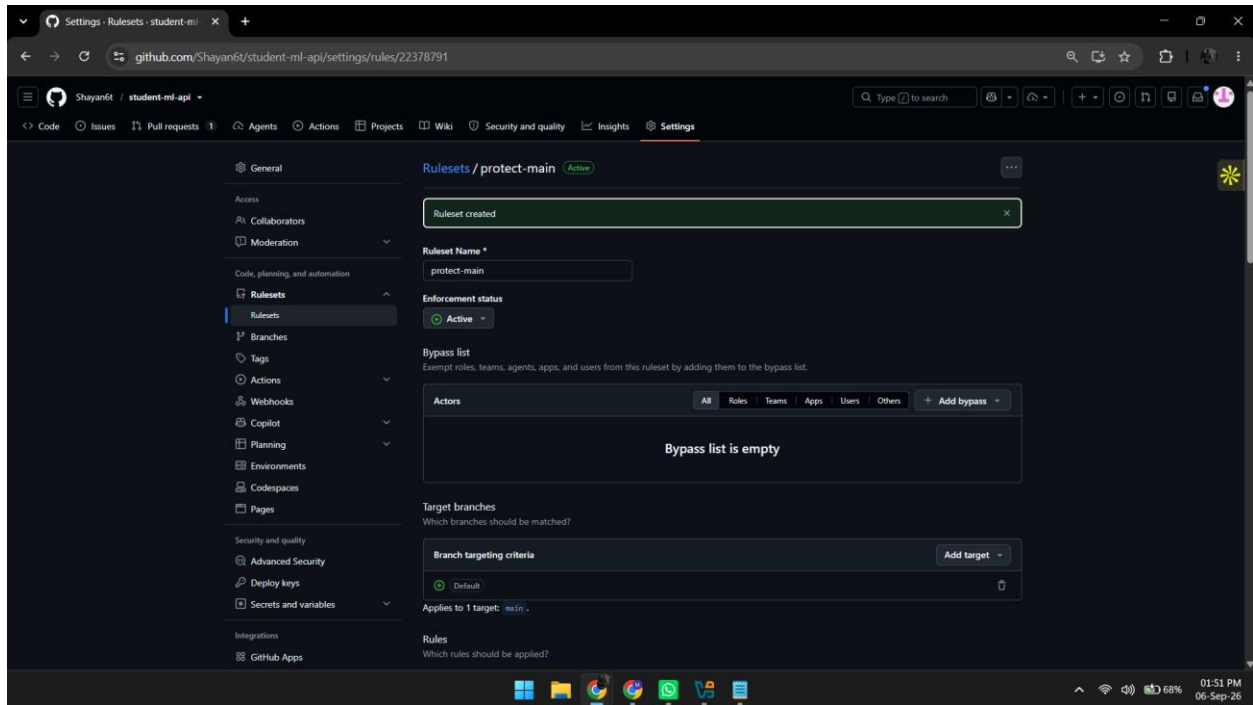


Figure 5: Initial CI pipeline execution triggered via GitHub Actions.

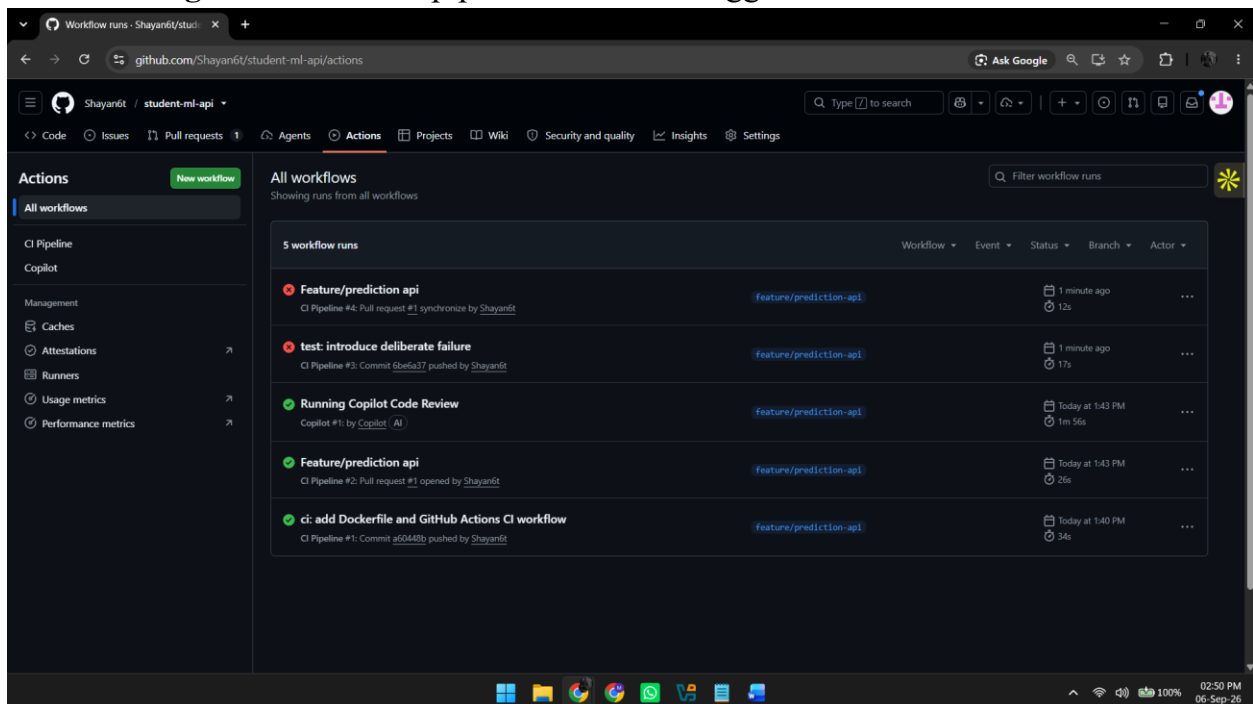


Figure 6: CI pipeline successfully blocking a PR due to a deliberate test failure.

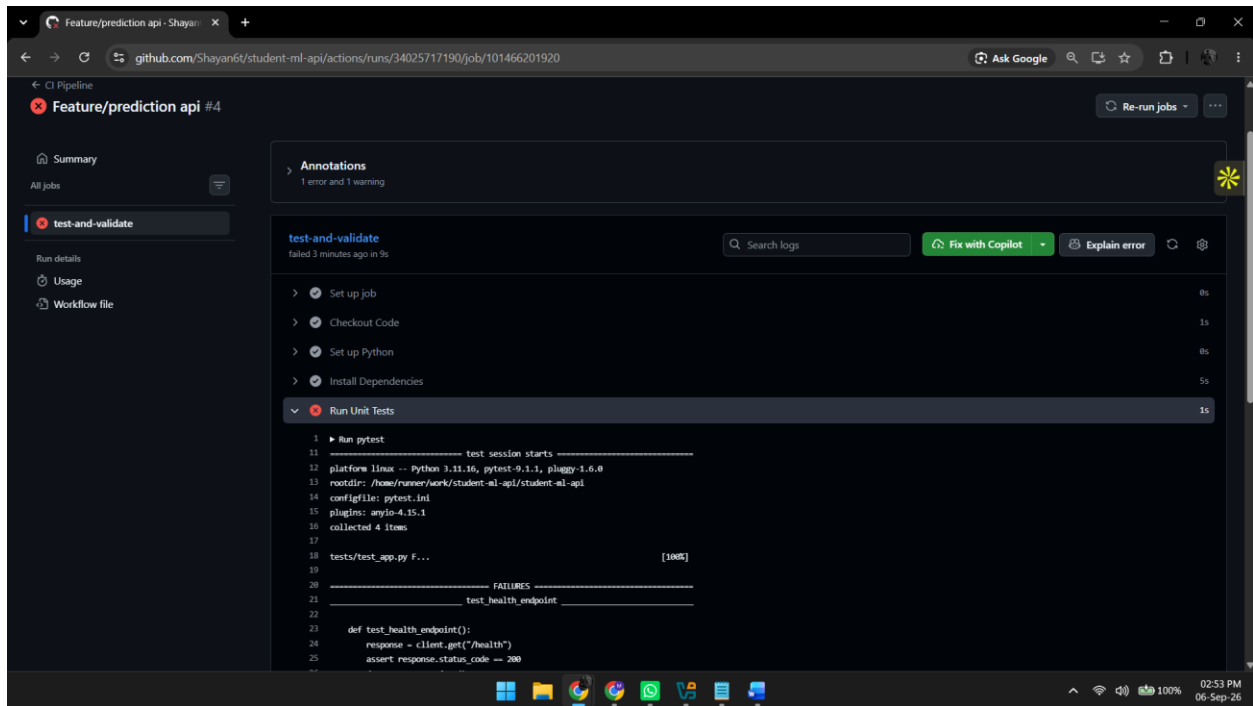


Figure 7: CI pipeline passing after the deliberate test failure was corrected.

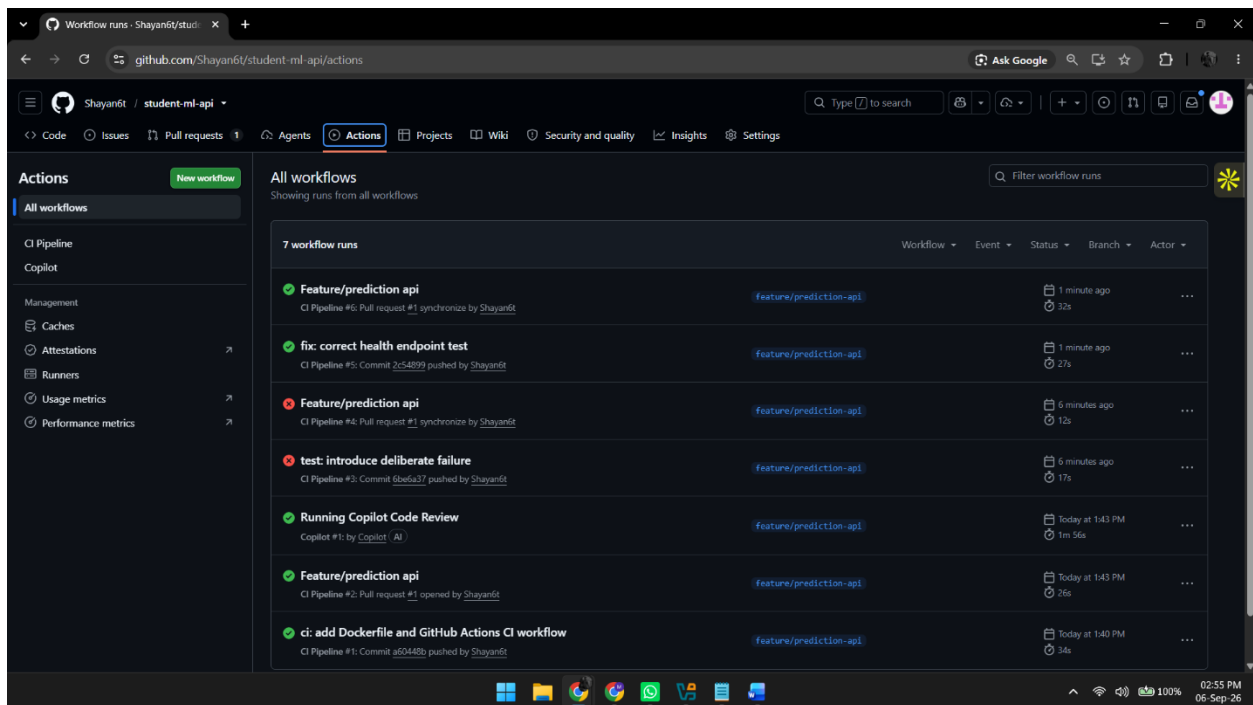


Figure 8: Detailed execution logs of the successful test-and-validate job.

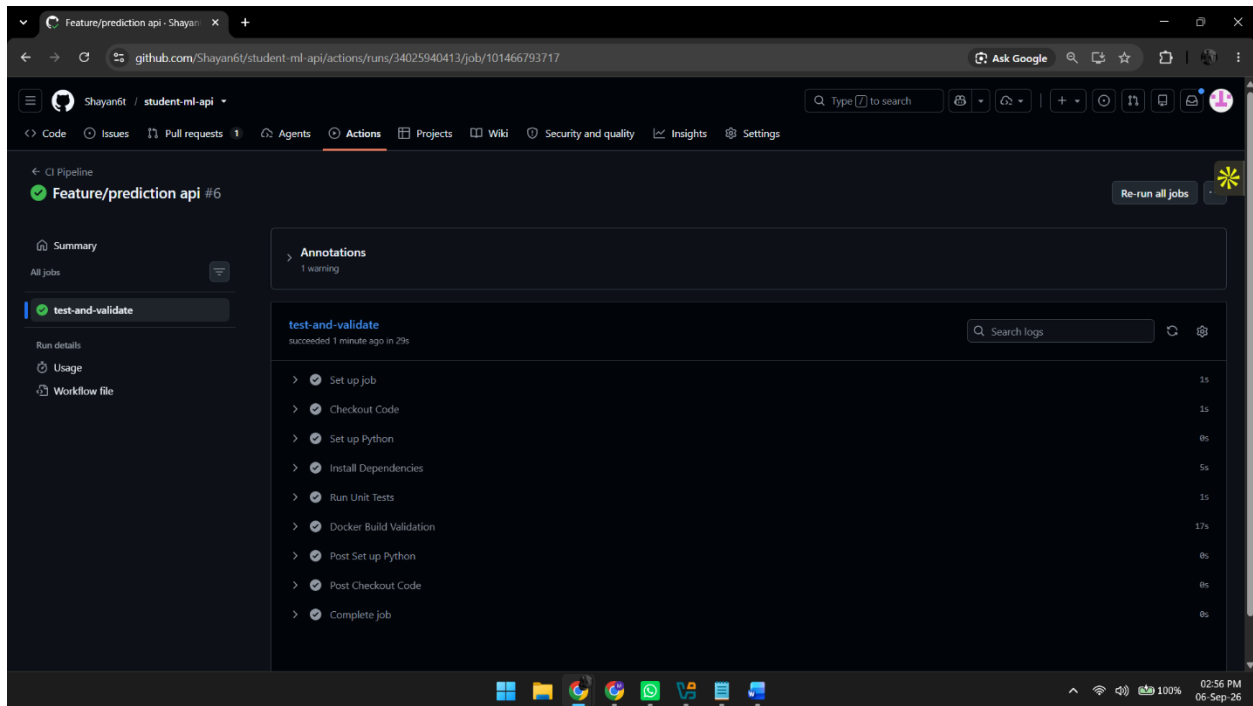
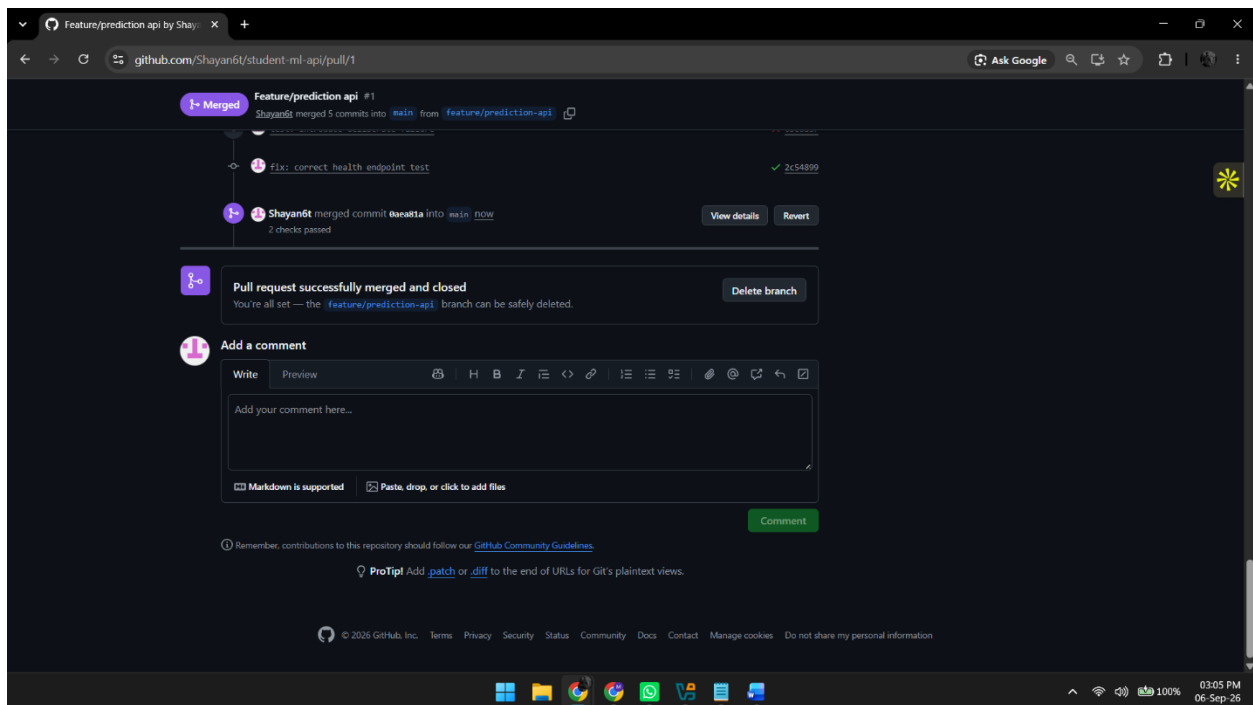
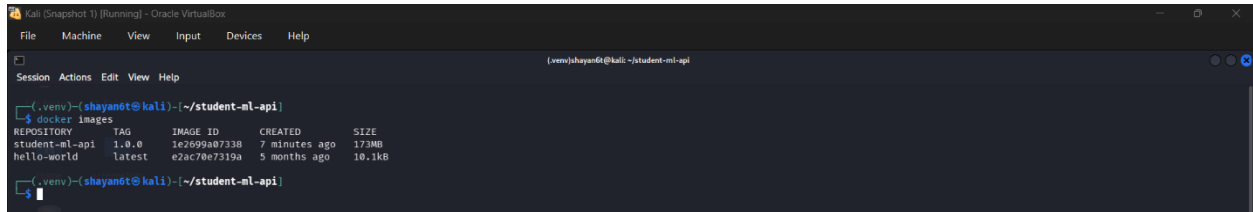


Figure 9: Successful merge of the first feature branch via Pull Request.



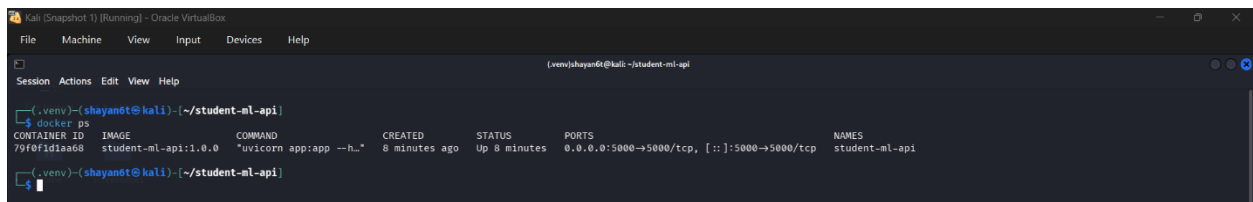
Parts 10–11: Local Container Inspection

Figure 10: Verification of the locally built **student-ml-api:1.0.0** Docker image.



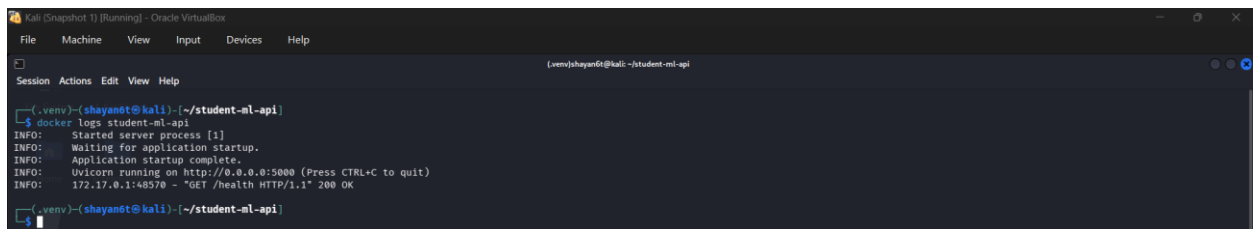
```
Kali (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
(.venv)shayan6t@kali: ~/student-ml-api
Session Actions Edit View Help
(.venv)~(shayan6t@kali)-[~/student-ml-api]
$ docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
student-ml-api 1.0.0     1e2699a07338   7 minutes ago  173MB
hello-world    latest    e2ac70e7319a   5 months ago   10.1kB
(.venv)~(shayan6t@kali)-[~/student-ml-api]
```

Figure 11: Container runtime confirmation showing port 5000 mapped to the host.



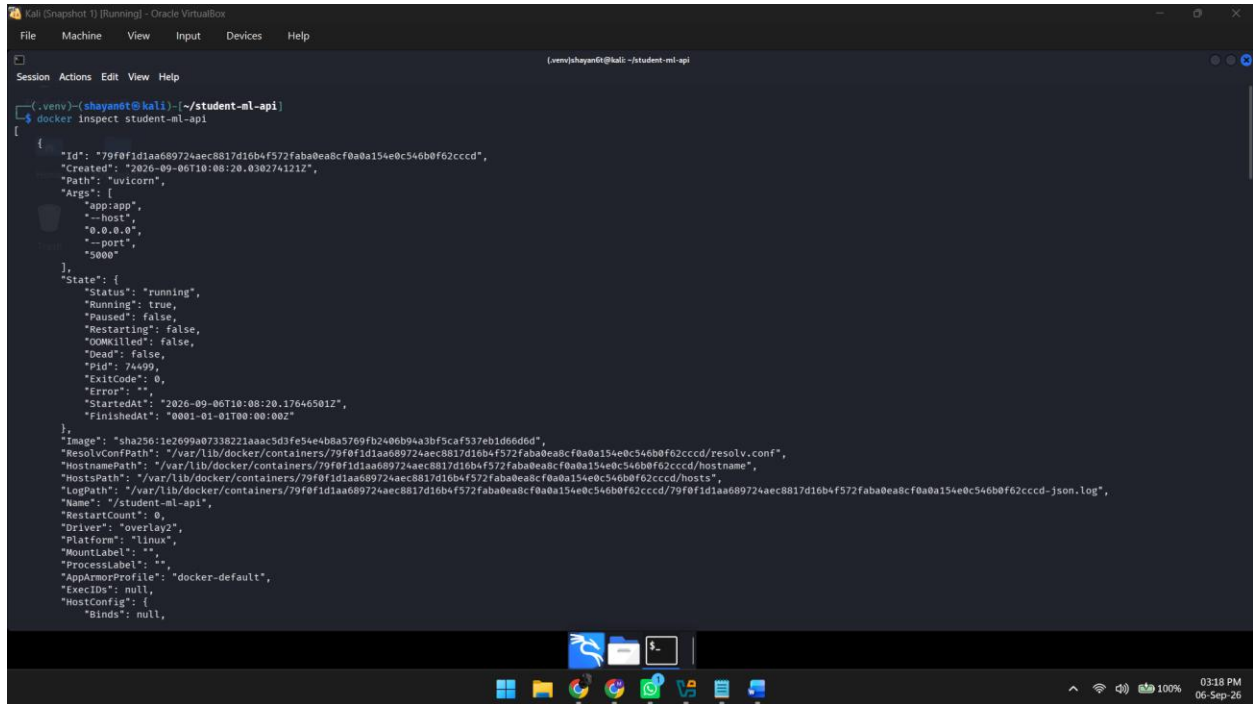
```
Kali (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
(.venv)shayan6t@kali: ~/student-ml-api
Session Actions Edit View Help
(.venv)~(shayan6t@kali)-[~/student-ml-api]
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
79f0f8daa68    student-ml-api:1.0.0  "uvicorn app:app --h-"   8 minutes ago  Up 8 minutes  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  student-ml-api
(.venv)~(shayan6t@kali)-[~/student-ml-api]
```

Figure 12: Container logs demonstrating successful Uvicorn server startup.



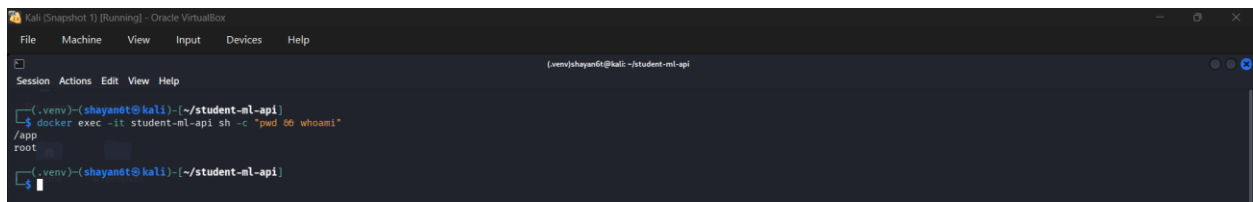
```
Kali (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
(.venv)shayan6t@kali: ~/student-ml-api
Session Actions Edit View Help
(.venv)~(shayan6t@kali)-[~/student-ml-api]
$ docker logs student-ml-api
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:5000 (Press CTRL+C to quit)
INFO: 172.17.0.1:48370 - "GET /health HTTP/1.1" 200 OK
(.venv)~(shayan6t@kali)-[~/student-ml-api]
```

Figure 13: Docker inspect output detailing container metadata and entrypoint.



```
[shayan@kali:~/student-ml-api]$ docker inspect student-ml-api
[
  {
    "Id": "79f0f1daa689724aec8817d16b4f572faba0ea8cf0a0a154e0c546b0f62cccd",
    "Created": "2026-09-06T10:00:20.030274121Z",
    "Path": "uvicorn",
    "Args": [
      "app:app",
      "--host",
      "0.0.0.0",
      "--port",
      "5000"
    ],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 74499,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-09-06T10:00:20.17646501Z",
      "FinishedAt": "0001-01-01T00:00:00Z"
    },
    "Image": "sha256:1e2699a07338221aac5d3fe54e4b8a5769fb2406b94a3bf5caf537eb1d66d6d",
    "ResolvConfPath": "/var/lib/docker/containers/79f0f1daa689724aec8817d16b4f572faba0ea8cf0a0a154e0c546b0f62cccd/resolv.conf",
    "HostnamePath": "/var/lib/docker/containers/79f0f1daa689724aec8817d16b4f572faba0ea8cf0a0a154e0c546b0f62cccd/hostname",
    "HostsPath": "/var/lib/docker/containers/79f0f1daa689724aec8817d16b4f572faba0ea8cf0a0a154e0c546b0f62cccd/hosts",
    "LogPath": "/var/lib/docker/containers/79f0f1daa689724aec8817d16b4f572faba0ea8cf0a0a154e0c546b0f62cccd/79f0f1daa689724aec8817d16b4f572faba0ea8cf0a0a154e0c546b0f62cccd-json.log",
    "Name": "/student-ml-api",
    "RestartCount": 0,
    "Driver": "overlay2",
    "Platform": "linux",
    "MountLabel": "",
    "ProcessLabel": "",
    "AppArmorProfile": "docker-default",
    "ExecIDs": null,
    "HostConfig": {
      "Binds": null,
```

Figure 14: Interactive shell execution (docker exec) verifying the working directory.



```
[shayan@kali:~/student-ml-api]$ docker exec -it student-ml-api sh -c "pwd 00 whoami"
/app
root
[shayan@kali:~/student-ml-api]$
```

Parts 12–16: Release Pipeline & GHCR

Figure 15: Release workflow failure caused by uppercase characters in the GHCR repository tag.

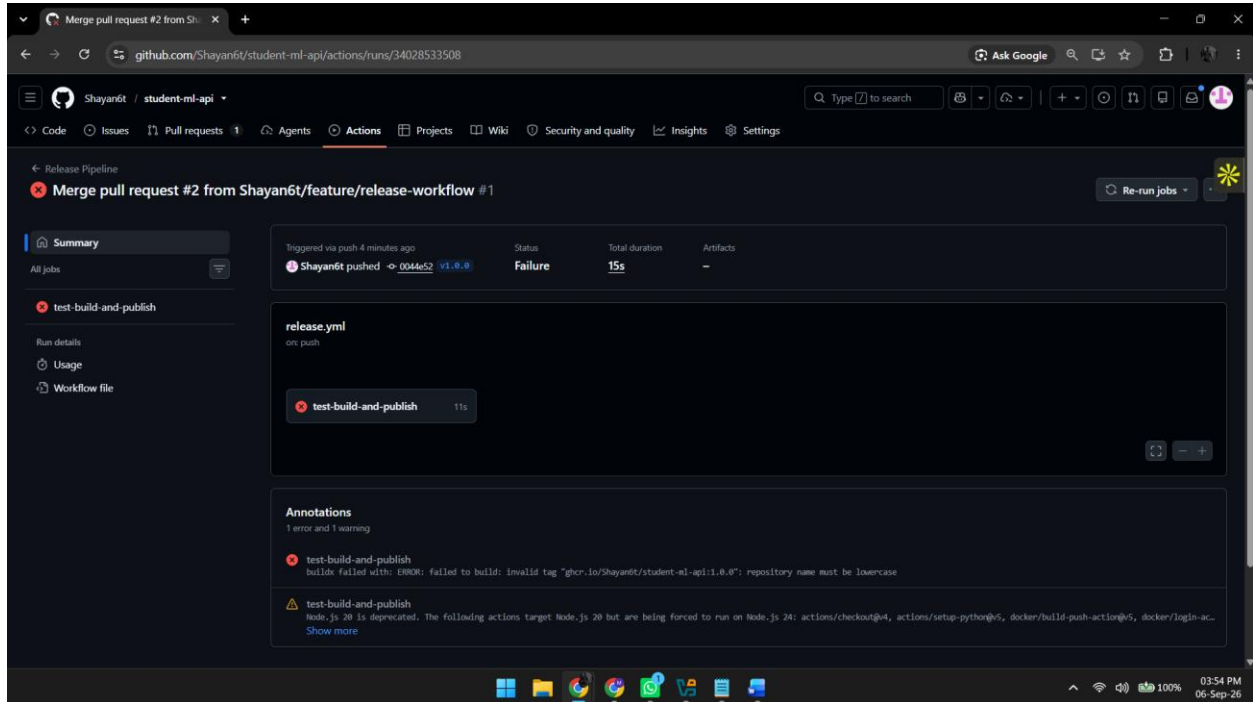


Figure 16: Successful Release workflow execution after sanitizing the repository name.

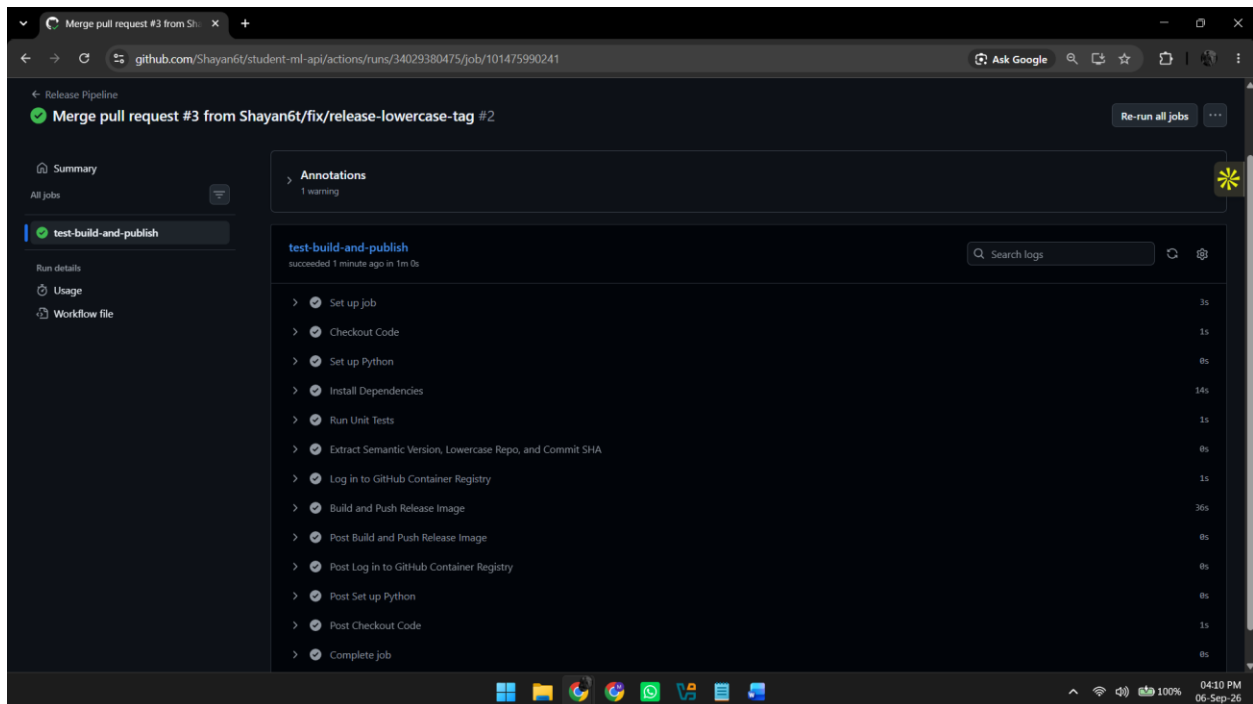
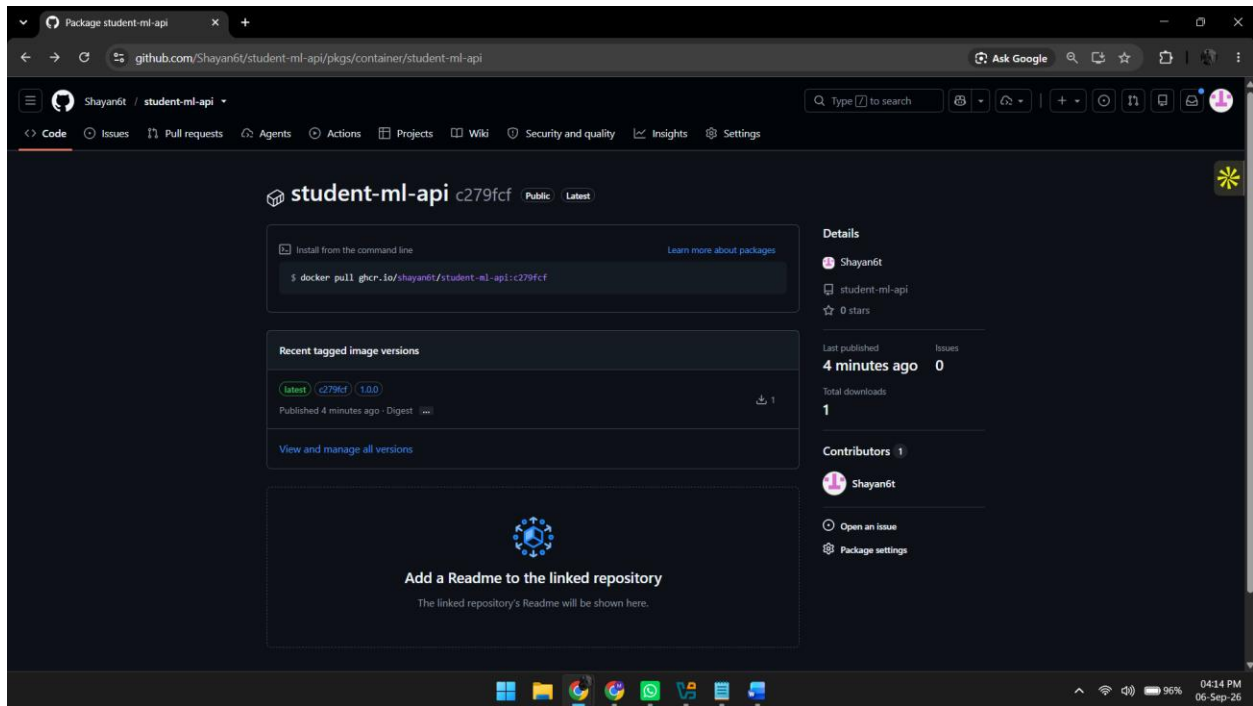
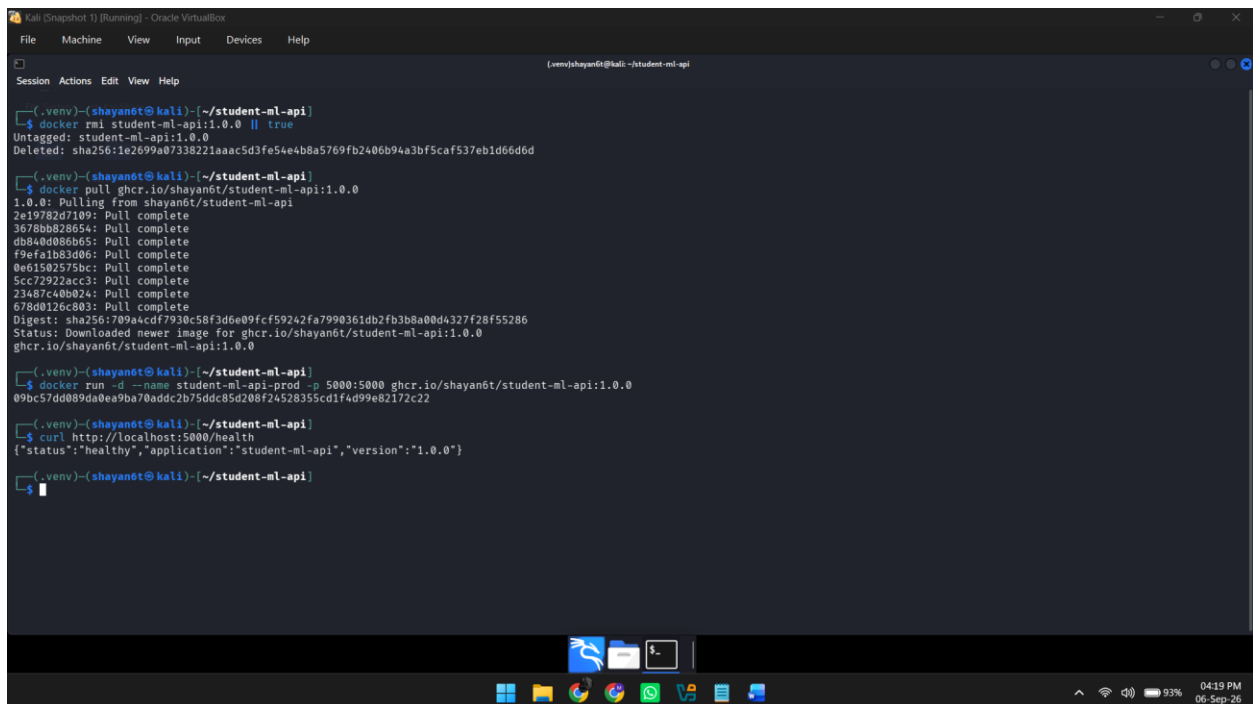


Figure 17: Published 1.0.0 image on GHCR featuring latest and commit SHA tags.



Part 17: Artifact Portability

Figure 18: Artifact portability verified by running the image directly from GHCR.



Parts 18–19: Iterative Release (v1.1.0)

Figure 19: Merged Pull Request for the v1.1.0 model metadata updates.

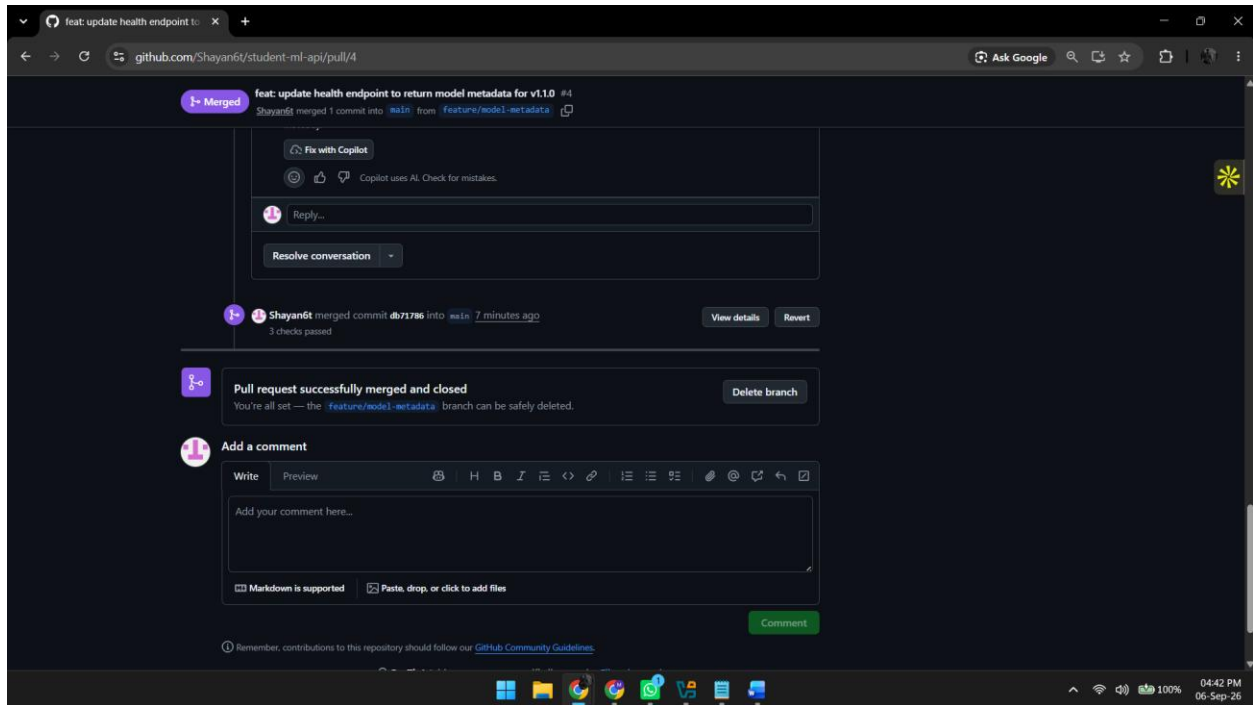
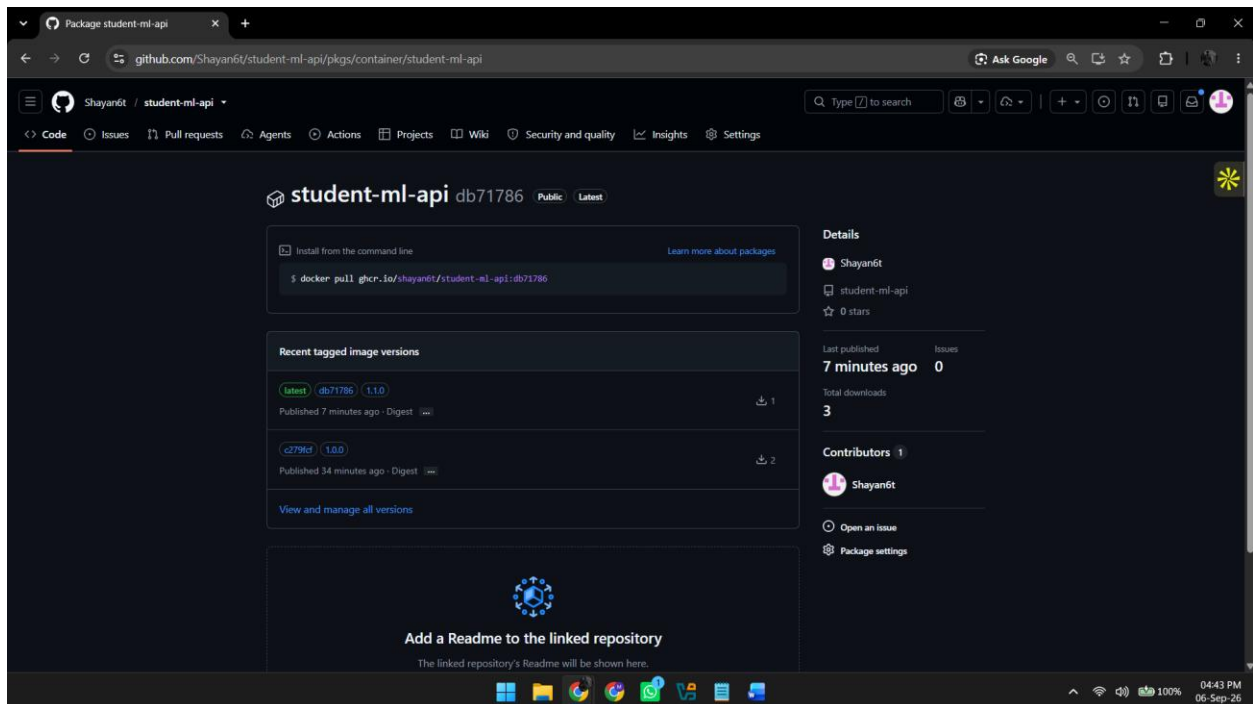
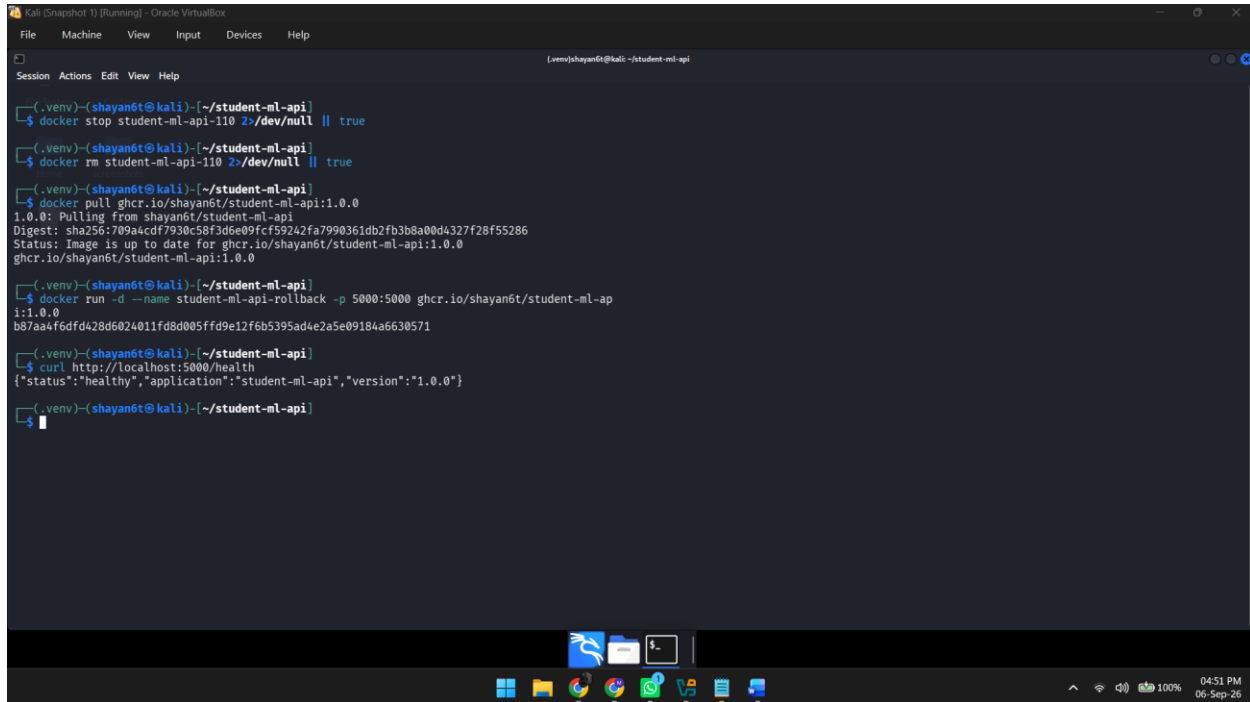


Figure 20: GHCR registry displaying the published 1.1.0 tag alongside the preserved 1.0.0 image.



Parts 20: Rollback Exercise

Figure 21: Instant disaster recovery demonstrated by rolling back the container to version 1.0.0 via GHCR.



```
Kali (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

(.venv)-(shayan6t@kali)-[~/student-ml-api]
$ docker stop student-ml-api-110 2>/dev/null || true

(.venv)-(shayan6t@kali)-[~/student-ml-api]
$ docker rm student-ml-api-110 2>/dev/null || true

(.venv)-(shayan6t@kali)-[~/student-ml-api]
$ docker pull ghcr.io/shayan6t/student-ml-api:1.0.0
1.0.0: Pulling from shayan6t/student-ml-api
Digest: sha256:709a4cdf7930c58f3d0e09fcf59242fa7990361db2fb3b8a00d4327f28f55286
Status: Image is up to date for ghcr.io/shayan6t/student-ml-api:1.0.0
ghcr.io/shayan6t/student-ml-api:1.0.0

(.venv)-(shayan6t@kali)-[~/student-ml-api]
$ docker run -d --name student-ml-api-rollback -p 5000:5000 ghcr.io/shayan6t/student-ml-api:1.0.0
b87aa4f6dfd428d6024011fd8d005ffd9e12f6b5395ad4e2a5e09184a6630571

(.venv)-(shayan6t@kali)-[~/student-ml-api]
$ curl http://localhost:5000/health
{"status":"healthy","application":"student-ml-api","version":"1.0.0"}

(.venv)-(shayan6t@kali)-[~/student-ml-api]
$
```

Part 21: Traceability Challenge

- **Pull Request Number:** #4
- **Merge Commit SHA:** db717868a1426b69a7164d025d7ad8b994b4a45e
- **Git Tag:** v1.1.0
- **Docker Image Tag:** ghcr.io/shayan6t/student-ml-api:1.1.0
- **Docker Image Digest:**
sha256:d5b08fd60b9da7e477523ed833e823093f296ee5eb4bf5f5d415905135e2fa3e

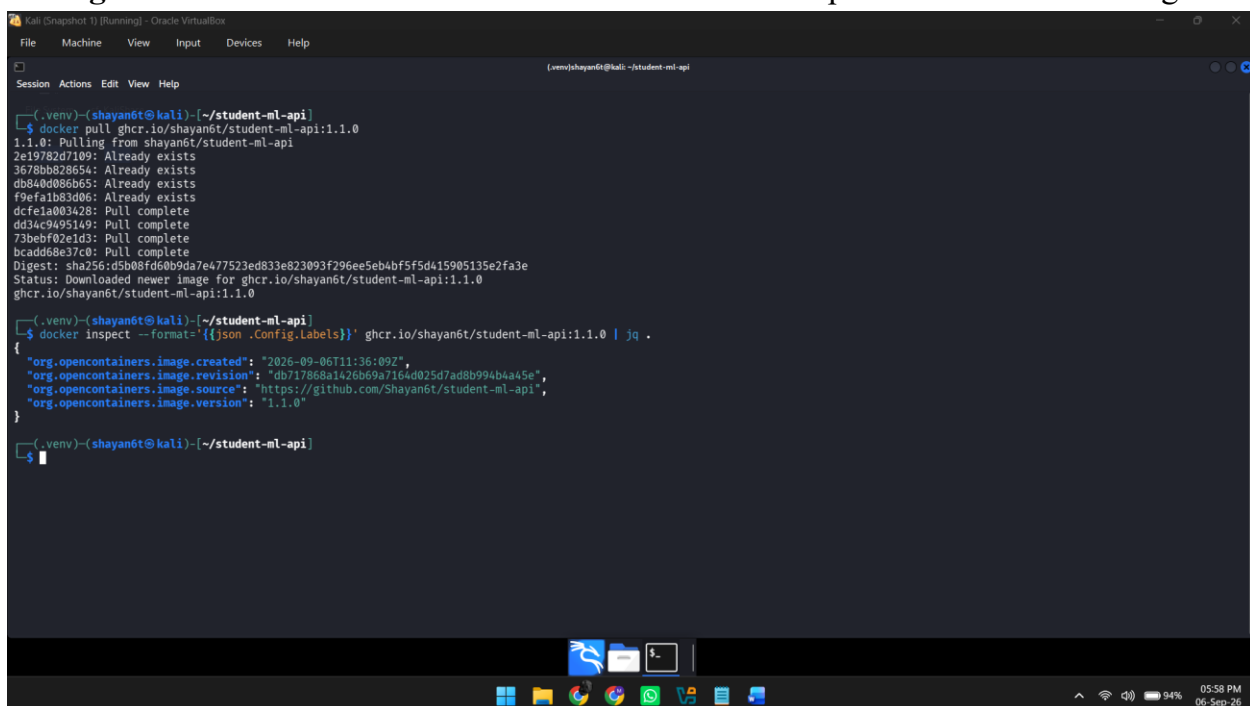
Part 22: Advanced GitHub Actions Requirement

Question: Why is publishing Docker images from every Pull Request bad practice?

- **Wastes Registry Storage:** Pull requests involve constant trial, error, and bug fixes. Uploading an image for every single commit fills up the registry with temporary "junk" images that no one will ever use.
- **Unreviewed and Risky Code:** Code inside a Pull Request hasn't been approved yet. If we push an image to the registry before peer review, someone might accidentally deploy untested or insecure code.
- **Accidental Deployments:** If feature branch images sit in the same registry as official versions, developers or deployment scripts could mistake an unfinished PR image for a stable production release.
- **Separation of Roles:** CI on a Pull Request should only verify and test the code to ensure nothing is broken. Publishing an official image should only happen after code is approved, merged to main, and marked with a version tag.

Part 23: Advanced Challenge: Image Metadata

Figure 22: Extraction of OCI metadata labels from the published container image.



```
kali (Snapshot 1) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

[venv]shayan6t@kali: ~/student-ml-api
Session Actions Edit View Help

[.venv]-(shayan6t@kali)-[~/student-ml-api]
└─$ docker pull ghcr.io/shayan6t/student-ml-api:1.1.0
1.1.0: Pulling from shayan6t/student-ml-api
2e19782d7109: Already exists
3678bb828654: Already exists
db840d086b5: Already exists
f9fa1b83d06: Already exists
dcfe1a003428: Pull complete
dd34c9495149: Pull complete
73bebf02e1d3: Pull complete
bcadd68e37cb: Pull complete
Digest: sha256:d5b08fd60b9da7e477523ed833e823093f296ee5eb4bf5fd415905135e2fa3e
Status: Downloaded newer image for ghcr.io/shayan6t/student-ml-api:1.1.0
ghcr.io/shayan6t/student-ml-api:1.1.0

[.venv]-(shayan6t@kali)-[~/student-ml-api]
└─$ docker inspect --format='{{json .Config.Labels}}' ghcr.io/shayan6t/student-ml-api:1.1.0 | jq .
{
  "org.opencontainers.image.created": "2026-09-06T11:36:09Z",
  "org.opencontainers.image.revision": "db717868a1426b69a7164d025d7ad8b994b4a45e",
  "org.opencontainers.image.source": "https://github.com/Shayan6t/student-ml-api",
  "org.opencontainers.image.version": "1.1.0"
}
```

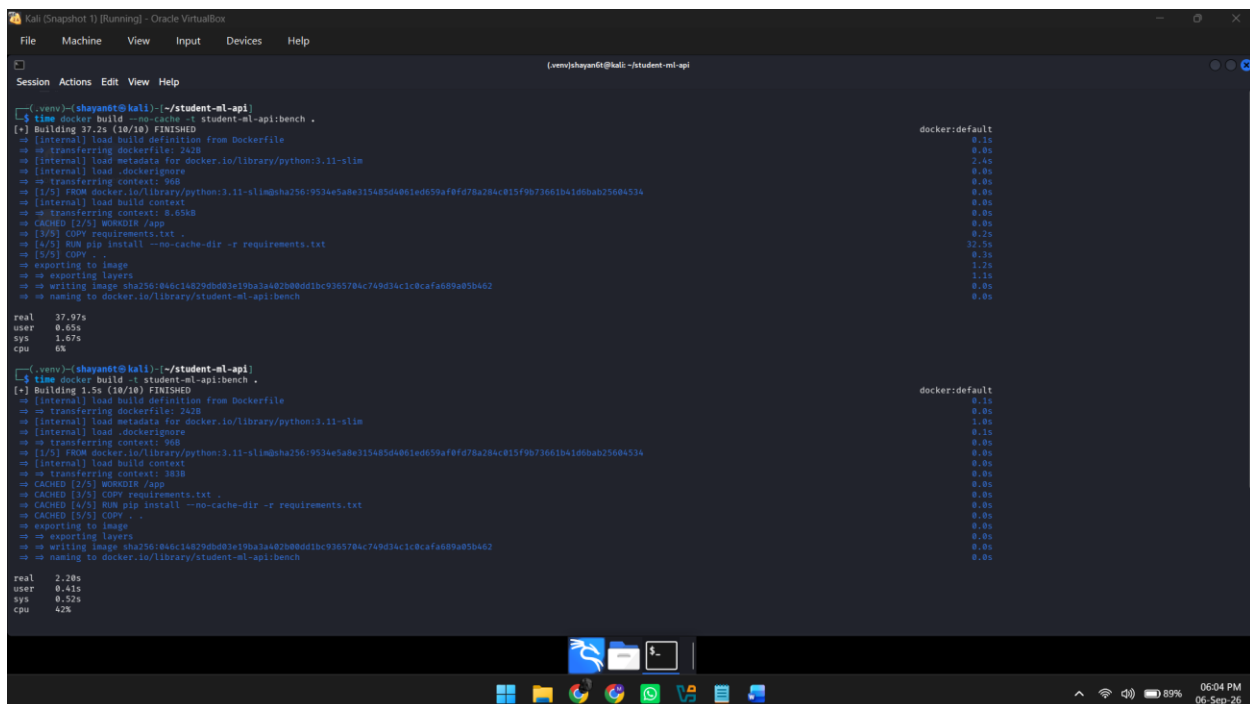
Part 24: Advanced Challenge: Commit SHA Tag

Question: Why is tagging with the commit SHA essential when tags like :latest are mutable?

Tags like :latest or :1.1.0 can be overwritten if an image is rebuilt and repushed. Tagging with the Git commit SHA (db71786) guarantees an immutable reference that connects the container binary directly to the exact source code commit used to compile it, preventing ambiguities about which exact codebase is running in production.

Part 25: Docker Build Cache

Figure 23: Docker layer caching efficiency demonstrated by comparing uncached and cached build times.



```
[venv]-(shayan@kali)-[~/student-ml-api]
└─$ time docker build --no-cache -t student-ml-api:bench .
[+] Building 37.2s (18/18) FINISHED
    => [internal] load build definition from Dockerfile
    => => transferring dockerfile: 242B
    => [internal] load metadata for docker.io/library/python:3.11-slim
    => [internal] load .dockerignore
    => => transferring context: 96B
    => [1/5] FROM docker.io/library/python:3.11-slim@sha256:9534e5ade315485d4061ed059af0fd78a284c815f9b73661ba160ba25004534
    => [internal] load build context
    => => transferring context: 8.65kB
    => CACHED [2/5] WORKDIR /app
    => CACHED [3/5] COPY requirements.txt
    => [4/5] RUN pip install --no-cache-dir -r requirements.txt
    => [5/5] COPY .
    => exporting to image
    => => exporting layers
    => writing image sha256:06dc14829dd082e19b3e1a4d7000d1bc936578ac749d34c1c0cfa609a05b462
    => => naming to docker.io/library/student-ml-api:bench

real    37.97s
user    0.55s
sys     1.67s
cpu      6%

[venv]-(shayan@kali)-[~/student-ml-api]
└─$ time docker build -t student-ml-api:bench .
[+] Building 1.5s (18/18) FINISHED
    => [internal] load build definition from Dockerfile
    => => transferring dockerfile: 242B
    => [internal] load metadata for docker.io/library/python:3.11-slim
    => [internal] load .dockerignore
    => => transferring context: 96B
    => [1/5] FROM docker.io/library/python:3.11-slim@sha256:9534e5ade315485d4061ed059af0fd78a284c815f9b73661ba160ba25004534
    => [internal] load build context
    => => transferring context: 181B
    => CACHED [2/5] WORKDIR /app
    => CACHED [3/5] COPY requirements.txt
    => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
    => CACHED [5/5] COPY .
    => exporting to image
    => => exporting layers
    => writing image sha256:06dc14829dd082e19b3e1a4d7000d1bc936578ac749d34c1c0cfa609a05b462
    => => naming to docker.io/library/student-ml-api:bench

real    2.28s
user    0.41s
sys     0.52s
cpu     42%
```

Explanation:

- **Layer Caching:** Docker computes checksums of instructions and file contexts. When dependencies in requirements.txt remain unchanged, Docker reuses the cached intermediate layers (CACHED [4/5] RUN pip install...), cutting build times dramatically.
- **Ordering Matters:** Placing COPY requirements.txt . and RUN pip install ... *before* COPY . . ensures that code modifications do not invalidate the slow dependency installation layer.

Part 26: Failure Analysis

Incident Scenario	Observed Symptom	Root Cause	Corrective Action Applied
Deliberate CI Test Failure	test-and-validate job failed on PR #1 (exit code 1).	Pytest assertion failed: expected status == "wrong", returned "healthy".	Corrected assertion in tests/test_app.py back to data["status"] == "healthy" and pushed fix.

Evidence:

Run Unit Tests15

```
1 ▶ Run pytest
11 ===== test session starts =====
12 platform linux -- Python 3.11.16, pytest-9.1.1, pluggy-1.6.0
13 rootdir: /home/runner/work/student-ml-api/student-ml-api
14 configfile: pytest.ini
15 plugins: anyio-4.15.1
16 collected 4 items
17
18 tests/test_app.py F... [100%]
19
20 ===== FAILURES =====
21 _____ test_health_endpoint _____
22
23 def test_health_endpoint():
24     response = client.get("/health")
25     assert response.status_code == 200
```

Incident Scenario	Observed Symptom	Root Cause	Corrective Action Applied
Registry Case Sensitivity Error	test-build-and-publish failed during Build and Push Release Image step.	GHCR URLs strictly require lowercase characters. \${{ github.repository_owner }} evaluated to Shayan6t.	Added shell conversion step tr '[:upper:]' '[:lower:]' in release.yml.

Evidence:

release.yml

on: push

test-build-and-publish11s

Annotations

1 error and 1 warning

test-build-and-publish

buildx failed with: ERROR: failed to build: invalid tag "ghcr.io/Shayan6t/student-ml-api:1.0.0": repository name must be lowercase